



Tarea Módulo 1

Reconocimiento de
Elementos en el Desarrollo
de un Programa
Informático

Por Ángel Carneros



Índice

Parte 1. Explicación de los conceptos básicos:	3
<i>Describe en tus propias palabras qué es el código fuente, código objeto y código ejecutable.</i>	3
- Código fuente	3
- Código objeto	3
- Código ejecutable	3
- Fases	
- Ejemplos	
 2. Clasificación de lenguajes de programación:	 9
<i>Según el nivel de abstracción</i>	9
<i>Según el paradigma de programación</i>	10
	12
Parte 2. Actividad práctica y de análisis	13
Identificación de paradigmas de programación a partir de ejemplos:	13
Actividad en grupo	13



Parte 1: Análisis teórico de conceptos

1. Explicación de los conceptos básicos:

Describe en tus propias palabras qué es el código fuente, código objeto y código ejecutable.

Código fuente: Es el programa exactamente como lo escribe el programador en un lenguaje de alto nivel el cual las personas podemos leer y entender. Se guarda en ficheros y es lo que se edita, se corrige y se mantiene. El procesador no puede ejecutarlo directamente.

Código objeto: Es el resultado de pasar el código fuente por un compilador. Ya está traducido a lenguaje máquina pero todavía no es ejecutable, es un fichero intermedio que puede tener referencias sin resolver a funciones o librerías que están en otros ficheros. Cada fichero fuente suele generar su propio código objeto.

Código ejecutable: Es el programa final y completo. Se obtiene cuando el enlazador une los distintos códigos objeto y las librerías necesarias resolviendo todas las referencias. El resultado puede cargarse en la memoria y ser ejecutado por el sistema operativo.



Explica las fases por las que pasa un programa desde que es escrito en un lenguaje de programación hasta que es ejecutado en el procesador.

Un programa se escribe en un lenguaje de alto nivel, que el procesador no entiende directamente. Por eso necesita un proceso de traducción, la compilación, hasta obtener un código que pueda entrar en la CPU.

1. Código fuente: es el código que el programador escribe en el lenguaje de programación elegido (Java, C++, etc.).

2. Compilación: el compilador traduce el código fuente y genera el código objeto. Para ello pasa por estas fases:

- **Análisis léxico:** comprueba que los elementos que aparecen en el archivo pertenecen al lenguaje de programación utilizado.
- **Análisis sintáctico:** determina si los elementos que llegan del analizador léxico vienen en el orden correcto.
- **Análisis semántico:** determina si las sentencias escritas por el programador tienen sentido.
- **Generación de código intermedio (opcional):** cuando terminan los análisis se supone que no hay errores, y se genera una representación intermedia independiente del procesador en el que se ejecutará el programa.
- **Optimización de código intermedio (opcional):** intenta mejorar el código generado para conseguir mejor rendimiento.
- **Generación de código final:** se genera el código objeto, que ya depende del conjunto de instrucciones de la CPU utilizada.



3. Código objeto: es el resultado de finalizar el proceso de compilación.

4. Enlazado y código ejecutable: el código objeto se enlaza con las librerías necesarias y se obtiene el código ejecutable, que es el único que puede entrar en el procesador para ser ejecutado.

5. Carga y ejecución: cuando se va a ejecutar, el sistema operativo carga el programa en la memoria principal (RAM). Una vez ahí se carga en la CPU, formada por la ALU, los registros y la unidad de control siguiendo la arquitectura Von Neumann.



Usa ejemplos concretos para cada concepto, mencionando en qué fase interviene cada uno en el desarrollo de un programa (análisis léxico, sintáctico, semántico, etc.).

Un programa se escribe en un lenguaje de alto nivel, que el procesador no entiende directamente. Por eso necesita un proceso de traducción, la compilación, hasta obtener un código que pueda entrar en la CPU.

Ejemplo:

```
c  
  
    int main() {  
  
        int a = 5;  
  
        int b = a + 3;  
  
        printf("%d", b);  
  
        return 0;  
  
    }
```



- 1. Código fuente:** Es el texto anterior, escrito por el programador en un lenguaje de alto nivel. El procesador no puede ejecutarlo directamente.
- 2. Análisis léxico:** Comprueba que los elementos del archivo pertenecen al lenguaje. En `int b = a + 3;` reconoce `int` (palabra reservada), `b` y `a` (identificadores), `=` y `+` (operadores), `3` (número) y `;` (fin de sentencia). Error de esta fase: `int b = a + 3 @;` → el símbolo `@` no pertenece al lenguaje.
- 3. Análisis sintáctico:** Determina si los elementos que vienen del analizador léxico están en el orden correcto. `int b = a + 3;` es válido porque sigue la estructura tipo + nombre = expresión. Error de esta fase: `int b = a + ;` → todos los símbolos existen, pero falta un operando después del `+`.
- 4. Análisis semántico:** Determina si las sentencias tienen sentido, aunque estén bien escritas. Error de esta fase: `int b = c + 3;` → `c` no ha sido declarada. Otro ejemplo: `int a = "hola";` → se intenta guardar un texto en una variable entera.
- 5. Generación de código intermedio (opcional):** Cuando los análisis terminan sin errores, se crea una representación independiente del procesador.

Por ejemplo:

$t1 = a + 3$

$b = t1$

Esta misma representación sirve para distintas CPU.



6. Optimización de código intermedio (opcional): Mejora el código para que rinda más. Como a vale 5, el compilador puede calcular $5 + 3$ en compilación y dejar directamente $b = 8$, de modo que el programa final se ahorra una suma.

7. Generación de código final a código objeto: Se traduce a instrucciones máquina, que ya dependen del conjunto de instrucciones de la CPU. Por ejemplo, el mismo programa produce código distinto para un procesador x86 que para uno ARM. El resultado es el código objeto con un fallo aún pendiente: la llamada a `printf` no está resuelta porque esa función no está escrita en nuestro fichero.

8. Enlazado a código ejecutable: El código objeto se enlaza con las librerías necesarias. Aquí se une con la librería estándar de C, que contiene `printf`. El resultado es el código ejecutable, el único que puede "entrar" en el procesador. Error de esta fase: si la librería no está disponible, el enlazador da un error de referencia sin definir para `printf`.

9. Carga y ejecución: El sistema operativo carga los datos y las instrucciones del ejecutable en la memoria RAM. Después pasan a la CPU (arquitectura Von Neumann): la unidad de control gestiona las instrucciones, los registros guardan a y b, y la ALU realiza la suma $5 + 3$. La salida por pantalla es 8.



2. Clasificación de lenguajes de programación:

Investiga y clasifica los lenguajes de programación en función de:

- Nivel de abstracción: alto, medio y bajo.
- Paradigma de programación: imperativo y declarativo.

Según el nivel de abstracción

El nivel de abstracción indica cuántos detalles del hardware oculta el lenguaje al programador. Cuanto más alto es, menos necesita conocer, por ejemplo, el conjunto de instrucciones máquina de la CPU.

NIVEL	Características	Ejemplos
Alto	Cercano al lenguaje natural, más fácil de aprender y usar	Java (T), C++ (T), Kotlin (T), Python, JavaScript, C#, Swift, PHP (T)
Medio	Combina construcciones de alto nivel con acceso cercano al hardware, con mejor rendimiento	C (T), Fortran (T)
Bajo	Depende de las características de cada máquina y de su procesador	Ensamblador (T), lenguaje máquina



Según el paradigma de programación

Paradigma	Qué hace	Subparadigmas	Ejemplos
Imperativo	Crea algoritmos mediante instrucciones y expresiones: se indica cómo llegar al resultado	Orientado a objetos, procedimental	POO: Java (T), C++ (T), C#. Procedimental: C (T), Pascal, Fortran (T)
Declarativo	Solo especifica qué resultado se espera, sin indicar los pasos; se expresan relaciones lógicas y matemáticas	Funcional, lógico	Funcional: Haskell (T), Lisp. Lógico: Prolog. Consultas: SQL



Da al menos dos ejemplos de lenguajes para cada categoría y explica brevemente por qué pertenecen a esa clasificación.

Según el nivel de abstracción

Alto nivel

- **Java:** oculta al programador el conjunto de instrucciones de la CPU y se acerca al lenguaje natural. Además es independiente de la plataforma, lo que lo hace adecuado para un portal web o apps móviles.
- **Kotlin:** se usa para apps Android y no exige conocer detalles del hardware. Es un lenguaje moderno con funciones lambda y otras facilidades para el programador.
- **C++ y Python:** también de alto nivel, porque ofrecen abstracciones como clases y estructuras de datos que ocultan el funcionamiento de la máquina.

Nivel medio

- **C:** en su día se consideró de alto nivel, pero hoy se sitúa en el nivel medio porque los lenguajes han evolucionado. Es más cercano al hardware, por eso el caso práctico 2 lo propone para elementos que requieren mejor rendimiento.
- **Fortran:** el temario lo cita como ejemplo de nivel medio. Su diseño está orientado al cálculo y se acerca más a la máquina que los lenguajes de alto nivel actuales.



Bajo nivel

- **Ensamblador:** es el de nivel de abstracción más bajo, porque depende de las características de cada máquina. Cada procesador tiene el suyo, y el programador trabaja con instrucciones cercanas a las de la CPU.
- **Lenguaje máquina:** es el que entiende directamente el procesador, en ceros y unos. No hay ninguna abstracción y por eso es el más dependiente del hardware.

Según el paradigma de programación

Imperativo (se indica cómo llegar al resultado mediante instrucciones y expresiones)

- **Java** (orientado a objetos): el programa se organiza en clases y objetos, y se escriben las instrucciones paso a paso para resolver el problema.
- **C** (procedimental): el programa se escribe como una secuencia de instrucciones agrupadas en funciones que indican los pasos a seguir.
- **C++, Kotlin y Swift** también son imperativos. Kotlin y Swift incorporan además funciones lambda.



Parte 2: Actividad práctica y de análisis

1. Identificación de paradigmas de programación a partir de ejemplos:

Fragmento 1: Imperativo. Se indica paso a paso cómo se hace la suma, recorriendo la lista y sumando los números uno por uno.

Fragmento 2: Declarativo. Solo se dice qué resultado se quiere (los nombres de los empleados mayores de 30) sin explicar cómo se obtiene.

Fragmento 3: Declarativo. El factorial se define con una regla matemática (el de 0 es 1 y el de n es n por el factorial de $n-1$) sin detallar los pasos para calcularlo.

Fragmento 4: Imperativo. Se describen los pasos del proceso: recorrer la lista y comprobar cada producto uno por uno.

2. Actividad en grupo:

Descripción imperativa

1. Coger dos rebanadas de pan de molde.
2. Untar mantequilla en una cara de cada rebanada.
3. Poner una loncha de queso sobre una rebanada.
4. Poner una loncha de jamón encima del queso.
5. Poner otra loncha de queso encima del jamón.
6. Tapar con la otra rebanada, con la mantequilla hacia fuera.
7. Calentar una sartén a fuego medio.
8. Poner el sándwich en la sartén durante 2 minutos.
9. Darle la vuelta y dejarlo otros 2 minutos.



10. Retirarlo de la sartén y servirlo.

Descripción declarativa

Quiero un sándwich mixto de jamón y queso caliente con el pan tostado por los dos lados y el queso fundido.

Comparación

El enfoque imperativo tiene como ventaja que cualquiera puede seguirlo aunque no sepa cocinar, porque explica cada detalle del proceso. Como desventaja es que es más largo, cuesta más escribirlo y si cambia algo hay que reescribir los pasos.

El enfoque declarativo tiene como ventaja que es corto, claro y se centra en el resultado. Además, permite que quien lo haga elija la mejor forma de conseguirlo. Como desventaja, solo funciona si quien lo recibe ya sabe cómo hacerlo y no se controla el proceso así que el resultado puede variar según quién lo prepare.

En conclusión, igual que pasa en programación, ninguno es mejor que el otro: el imperativo es útil cuando necesitamos controlar cada paso y el declarativo cuando lo importante es el resultado y confiamos en que alguien sabe como conseguirlo.